

An Analysis of Process Characteristics for Developing Scientific Software

Diane Kelly, Royal Military College of Canada, Canada

ABSTRACT

The development of scientific software is usually carried out by a scientist who has little professional training as a software developer. Concerns exist that such development produces low-quality products, leading to low-quality science. These concerns have led to recommendations and the imposition of software engineering development processes and standards on the scientists. This paper utilizes different frameworks to investigate and map characteristics of the scientific software development environment to the assumptions made in plan-driven software development methods and agile software development methods. This mapping exposes a mismatch between the needs and goals of scientific software development and the assumptions and goals of well-known software engineering development processes.

Keywords: Extreme Programming, Professional End-User Developer, Scientific Software, Scientific Software Development, Software Development Process

1. INTRODUCTION

The methods for developing scientific software have recently garnered the attention of the software engineering community, resulting in several studies of scientists as “end-user developers”. Segal (2004) differentiated scientists as “professional” end-user developers, pointing out their extensive knowledge in a technical domain and their high comfort level in writing code. There have been suggestions (Ackroyd et al., 2008; Pitt-Francis et al., 2008; Segal, 2005; Wood & Kleb, 2003) that agile software development methodologies (Highsmith & Cockburn, 2001) rather than “plan-driven”

methodologies (Royce, 1970) are better suited to the development of scientific software.

This paper examines the characteristics of the environment in which scientists develop their own software and compares these to the assumptions and goals of plan-driven and agile software development methodologies. The conclusion is that there is a mismatch in both types of methodologies when applied to the development of scientific software.

We first define what we mean by scientific software and discuss characteristics of its development. In Section 3, we make initial observations about these characteristics and the assumptions inherent in plan-driven and agile software development methods. Section 4 uses a variety of models to further explore the

DOI: 10.4018/joeuc.2011100105

characteristics of scientific software development as opposed to inherent assumptions that underlie plan-driven and agile development methods. Section 5 summarizes our findings and Section 6 concludes.

2. SCIENTIFIC SOFTWARE

2.1. Definition of Scientific Software Development

A project that we categorize as scientific software development has the following three characteristics. First, the software is written to answer a scientific question. The question can be general, such as, “Is it safe to operate this nuclear generating station?” or specific, such as, “Can I track satellites with this telescope?” Second, the writing of the software necessitates the close involvement of someone with deep domain knowledge in the application area related to answering the question. The person with the deep domain knowledge, the scientist, usually writes the software. This is Segal’s professional end-user developer. We have observed cases where the scientist has not written the software, but the software developer has had to become enough of an expert in the domain to understand and implement what the scientist is describing. Third, the software provides output data to support the scientific initiative. A human is in the system loop to examine the data and make observations and ultimately, answer the scientific question. The expectation is that the data provided by the computer solution is correct and will not misguide the scientist answering the question.

The science application can be solving large systems of differential equations, analyzing immense amounts of data such as imagery or bioinformatics, and computing using analytical and empirical models. Our definition of scientific software, however, excludes the following: control software whose main functioning involves the interaction with other software and hardware; user interface software that may provide the input for and report of scientific

calculations; and any generalized tool that scientists may use in support of developing and executing their software, but does not of itself answer a scientific question.

Scientific software may have an extensive graphical user interface or interact with other software or hardware to obtain data. It may run on complex multi-processors and require middleware support to make use of hardware capabilities. But in the following discussion, we are talking exclusively about the code that implements the science application whether it is designed as a separate module or inserted into a product that includes these other parts. We contend that this core scientific part of the product requires different considerations when it is being developed.

2.2. Characteristics of Scientific Software

Several studies have looked at the characteristics that differentiate the environment and activities of scientific software development from the development of other types of software (Basili et al., 2008; Boisvert & Tang, 2001; Matthews et al., 2008; Post & Kendall, 2004; Sanders & Kelly, 2008; Segal, 2003, 2004). Some characteristics are invariant whether the software is developed by a scientist working alone (Sanders & Kelly, 2008) or a large distributed team (Matthew et al., 2008), or whether the code is a thousand lines of code or over a million.

The most noteworthy of the characteristics is the focus on the need for correctness, or trust, in the software. Some scientists stated baldly, “the software must never lie to me” (Sanders & Kelly, 2008). As explained by Kelly (2008) “This [trustworthiness] takes priority over efficiency, reliability, user-friendliness, cost-effectiveness, and portability ... If the software gives the wrong answer, all other qualities become irrelevant.” Hatton et al. (1994, 2007) and Miller (2006) both give examples of cases where correctness of the software was the sole factor in success or failure. Hatton describes an example of industrial software where the success of oil-exploration

businesses depended on the accuracy of the software output. Miller describes a case where a mistake in an example of research software forced the retraction of several research papers from well-regarded journals.

Although the word “correctness” is often used, we prefer the word “trustworthiness”. Hook and Kelly (2009) discuss correctness versus trustworthiness as follows. “Correctness is essentially a Boolean decision, either true or false. On the other hand, trust is on a graduated scale, ultimately determined by judgment. The level of trust can be different for every example of scientific software, and possibly different for each individual output data. When a scientist says that a computer output is acceptable within forty percent of benchmark data, he/she is talking about trust. Nothing can be said about the Boolean concept of correctness.” Granted, in Hatton’s (1994) and Miller’s (2006) examples, there was misplaced trust in the software. This leads into a long discussion of the difficulty of testing scientific software. Weyuker’s paper (1982) on testing non-testable software is a good start to that discussion.

Output from scientific software is the culmination of a series of interactions amongst scientific models, numerical techniques, data, source code, and hardware (Boisvert & Tang, 2001). Change of hardware platform or compiler option can alter the values of output. Similarly, different expressions in source code that mathematically are equivalent, can give different values of output, due to the fact that computer computations are neither associative nor distributive. Because of this, scientists are reluctant to make changes to source code or execution environments for any other reason than to improve the trust in their scientific output.

The complex interaction of scientific model, numerical technique, source code, data, and hardware affects the development environment. First, development of scientific software is a discovery process, requiring frequent iterations between coding and testing to discover the correct combination of the interacting

parts. Second, correct functioning of parts of the whole, although certainly desirable, in no way guarantees the correct functioning of the whole. Third, problems identified in the output of the software almost invariably require the scientist to participate in debugging. We’ve found that the scientist must be involved in any meaningful assessment of the software (Kelly et al., 2010).

The scientist is often both the developer and the user of the software, obviating the need for extensive documentation at any point during the development. At the other end of the spectrum, Sanders and Kelly (2008) describe how a customer can provide the scientific question to be answered, then relinquish almost all control of the project to the scientist who is in possession of the expertise best suited to make subsequent decisions. Again, our discussion here excludes the user interface; we have found the most successful scientific software projects assiduously include the customer during the phase of user interface design.

The goal of scientific software is to answer a technical question that generally cannot be answered otherwise. As a result, there is no half-measure in the development in the software. Either the software answers the question within acceptable error tolerances – and the software is trusted to do this – or it doesn’t. As well, the scope is indivisible in another way. For example, the scientific model cannot be implemented in one release and the solution technique saved for the next release.

Testing scientific software is almost wholly focused on “validation” testing (Sanders & Kelly, 2008; Boisvert & Tang, 2001). This is defined by the scientists as testing against real-world expectations, such as experimental results and monitored physical phenomena.

Characteristics of scientific software development are discussed further in the following sections, where the juxtaposition of the characteristic with assumptions associated with software development methods better illustrate the problems of mismatch.

3. INITIAL OBSERVATIONS ON SCIENTIFIC SOFTWARE AND DEVELOPMENT PROCESSES

3.1. Software Development Processes

Boehm and Turner (2005) divide software development processes roughly into two categories, plan-driven approaches and agile approaches. However, as noted by Segal (2005), software professionals generally tailor a methodology to their specific context, but need a base methodology that is a close fit to their situation. We provide a brief description of the two categories of development process, assuming these provide the base methodologies used by software professionals.

The original plan-driven approach, commonly known as the waterfall model, was first described by Royce (1970). There are several variations on this model (Pfleeger, 1998; Van Vliet, 2000). Royce's premise was to provide a mechanism of discussion, documentation, review, reflection, and testing that guaranteed that the code, when written, matched its requirements. The discovery process happens mostly when the requirements are elicited and specified. The subsequent steps in the process become more and more mechanical, such that the writing of the code should involve little or no discovery. This would otherwise involve an expensive (in both time and cost) backtracking, rewriting documents, and redoing reviews. The assumption is that requirements and design can be specified in detail and that knowledge of the system and its interactions is relatively complete before implementation. Quality assessment is focused on consistency of products against specifications. The scientist's focus on validation testing, the assessment of the finished product against real-world expectations seems to be in step with plan-driven practices.

Agile methodologies (Highsmith & Cockburn, 2001) were developed as an alternative to complete up-front planning. In agile methodologies, iterations are formulated into steps

that reduce planning and documentation to the minimum needed to deliver partial products in short time intervals. Agile methods are accompanied by sets of practices designed to fill the communication gap created by the reduction of documentation and up-front planning.

The two most popular agile development methodologies are Extreme Programming (XP) (Beck, 2003) and Scrum (Rawsthorne & Shimp, 2009).

The main focus of both Scrum and XP is a short time box, usually less than a month, in which a task is completed. The goal is to deliver a product on time. The accompanying assumption is that the product is readily divisible and testable.

One aspect of XP is the relationship of customer to developer. Jeffries et al. (2001) explain a key role of the customer where "the customer controls scope, deciding what to do and what to defer, to provide the best possible release by the due date" (p. 55). Often, the scientist is his/her own customer. On the other extreme, the customer provides the scientific question then defers to the scientist/developer to deal with details of the project. In between, there is a range of customers whose involvement depends on their understanding of the science. The assumption in XP is that the customer drives the development process. When applied to scientific software, the advantage of this is not always clear.

Beck (2000) discusses cost of change using XP, stating that "the cost of change [rises] slowly over time ... You [can] make big decisions as late in the process as possible" (p. 23). This type of thinking allows the extensive use throughout the project of refactoring, a key practice in XP. Our experience with developing scientific software tells us that some big decisions have to be made early. The choice of modeling technique, representation of data, discretization of equations, and so on, are key to the solution being pursued. In this, the development of scientific software takes on a flavour of plan-driven development.

3.2. Jackson's Problem Domain Versus Solution Domain

Jackson (1994, 2001) emphasizes the difference between problem domain and solution domain in developing software. He points out the need for the software developer to become "... expert in those aspects of the application [problem] domain that affect the design and construction of the software" (p. 251). The purpose of requirements engineering is to encourage the software developer to explore the problem domain. This has spawned considerable work in trying to find effective means of modeling and expression in requirements documents (Pfleege, 1998; Wiegers, 1999). The emphasis in plan-driven development models is on the need to explore the problem domain with the aim of reducing the amount of exploration when dealing with the solution domain and thus reducing costly rework. The approach is based on the statement that, "Many of the problems encountered in software development are attributable to shortcomings in the processes and practices used to gather, document, agree on, and alter the product's requirements" (Wiegers, 1999, p. 4).

One of the realities of scientific software development is the need to explore the solution domain because of the complex interactions amongst the different contributors to the computational solution. The exploration of the solution domain is far more extensive and critical to the project in relative terms than the exploration of the problem domain. In addition, the scientist is Jackson's "expert" in the problem domain. A sufficient understanding of the problem domain is already present in the scientist/developer. As far as expressing their understanding of the problem domain, scientists have a long history of precise vocabulary and expressions for documenting their needs and ideas. In the development of scientific software, the problem domain can be precisely captured by the scientist-expert in the domain, allowing attention to be turned relatively quickly to the solution domain where extensive time and attention are required.

4. USING MODELS TO EXPAND OUR UNDERSTANDING

To further our understanding of both the characteristics of scientific software development and appropriateness of existing software development approaches, we looked for frameworks that could be used for our comparisons. A requirement for these frameworks was to allow us to map the characteristics of scientific software development to assumptions underlying the different methodologies. The only fully suitable framework available was that of Boehm and Turner (2005). However, we made use of various other lists of items associated with development processes to perform a partial mapping and make additional observations.

4.1. Method Engineering

Literature on comparing software development methods, such as Song and Osterweil (1992) is heavily focused on the methods themselves and provides no characterization of the environment in which the method is to be used, nor the features of methods that would allow a mapping from environment to method.

Work in the area of method engineering, such as Rolland (1997) and Henderson-Sellers (2003) aims mostly at facilitating the construction of "situation-specific methods" (Rolland, 1997). Both the works cited are heavily involved in illustrating the ability and flexibility of their approaches so that all possible development methods can be modeled. Rolland (1997) describes how to evaluate *method chunks* as to their appropriateness for a given environment. Her description characterizes method chunks by intention (e.g., reengineer high-risk project), target (e.g., business process), and approach (e.g., domain-based approach). She characterizes environment by problem domain, complexity, and risk. The intention is to map entries in a library of method chunks to an environment. It's not clear how this mapping is to be done. Rolland also describes other work where projects are characterized by seventeen

Table 1. Five knowledge domains that contribute to the development and use of a software product

Knowledge Domain (KD)	Description
Physical world knowledge	Knowledge of physical world phenomena pertinent to the problem being solved.
Theory-based knowledge	Theoretical models that provide (usually) mathematical understanding and advancement of the problem towards a solution.
Software knowledge	Representations, conventions, and practices used to construct the solution.
Execution knowledge	Knowledge of the software and hardware tools used to create, maintain, control, and run the computer solution.
Operational knowledge	Knowledge related to the use of the computer solution.

contingency factors that are rated between high and low. Unfortunately, there is no description of analogous contingency factors for methods.

Henderson-Sellers (2003) describes how a selection of techniques can be mapped onto a method. However, the technique selection already presumes we have a mapping from the application environment to appropriate techniques and the paper is silent on how to get there.

4.2. Knowledge Domain Models

One point of interest in Rolland's paper (1997) is her reference to the "four worlds of IS engineering". She uses this as a framework to present the state-of-the-art in methods engineering, unfortunately with an inward focus on the methods themselves.

The idea of the "four worlds" framework can be extended to the five "knowledge domains" introduced by Kelly (2009). The five knowledge domains are summarized in Table 1 (adapted from Kelly, 2009).

To increase our understanding of the development of scientific software, we need to observe who has knowledge in these different domains. A further consideration is the impediments in transferring this knowledge across domains. Impediments include cultural, informational, and cognitive differences such as those described by Segal (2005). Impediments also include the "time at which knowledge is available", which is alluded to by the inclusion

of incremental software development in agile methodologies.

Because of the tight coupling of knowledge from the five domains in the final scientific software product, transfer of knowledge between domains is necessary to affect a working solution. Segal's discussion (2005) mainly deals with the boundary between theory-based knowledge and software knowledge. However, those designing the software still need to understand how physical world knowledge will be represented by the software solution and how the operation of the software will reflect on theory-based knowledge and possibly on interactions with decisions in the physical world. This points to a requirement for those building the software to be deeply knowledgeable about all five knowledge domains.

The acquisition of knowledge for a particular problem and its software solution can be considered on a time scale for all five knowledge domains, as progressing from general to specific knowledge. Depending on the purpose of the software solution, theory-based knowledge and physical world knowledge may be advancing as the operational knowledge of the final software product increases. This takes place on a relatively long time scale. On an intermediate time scale, theory-based knowledge, software knowledge, and execution knowledge increase as the developer/user better understands the vagaries of the new computer solution and its interplay with its underlying mathematics. The shortest time scale is the increase of software

knowledge related to the execution environment in order to get a minimally working product. It is this short time scale that is most aptly addressed by the agile development processes. The long time scale associated with operating the final product to understand the world it models, is more aptly associated with plan-driven processes. Both categories of software development processes miss the intermediate time scale activities, where the scientist is grappling with code, machines, and theory.

4.3. Iron Triangle Model for Software Project Management

The “iron triangle” of project management, first described by M. Barnes in 1969, provides a simple model to understand the parameters that affect the success or failure of a project. Even though the iron triangle model is now considered too simplistic (Pearson, 2009), nothing has yet replaced it and it corresponds well with the assumptions addressed by software development processes described in software engineering literature. Indeed, Beck (2000) begins his description of the use of Extreme Programming (XP), with a commonly enhanced version of the iron triangle. He adds “quality” as a fourth corner, or variable, to accompany the usual variables of scope (functionality), schedule, and cost.

Our observations on scientific software can be mapped onto this enhanced project management model.

Quality for scientific software is primarily “correctness”, or alternatively, trustworthiness. This immutable goal must be achieved. This variable is fixed in the project management plan for scientific software.

Scope is the scientific question that must be answered. This question is often difficult to divide, and so the scope is fixed.

We can argue that cost and schedule are trumped by scope and quality. Lateness of reports, theses, or decisions must be tolerated in order to achieve trustworthy and complete output from the software. The output data must be trusted to answer the scientific question. If

the scientific question cannot be answered by the software, then the journal paper cannot be written and the project not completed. A fixed-schedule-fixed-cost environment is very high-risk for the scientist since the only variable left to manage is scope, and this may be difficult to sub-divide.

We can observe that the almost overwhelming assumption of existing software development methodologies is that the fixed variables are cost and schedule, and that quality and scope are mutable. We contend that the fixed variables for scientific software are quality and scope, and that schedule must be mutable.

4.4. Boehm’s and Turner’s Model

Boehm’s and Turner’s Model (2005) for comparing and blending agile and plan-driven development methods appears to be the only such model described in the literature. They present “five critical factors involved in determining the relative suitability of agile or plan driven methods in a particular project” (p. 54) and observe that “... a project that is a good fit to agile or plan-driven for four of the factors, but not for the fifth, is a project in need of ... some mix of agile and plan-driven methods” (p. 54).

Segal (2005) discusses Boehm’s and Turner’s advice on melding agile with plan-driven methods in the context of developing scientific instrument software. Although this is not strictly within our definition of scientific software, their need to produce significant documentation for their long-lived software along with improved and more immediate communication suggests a blending of methods is necessary.

In order to improve our understanding why this is so, we discuss each of Boehm’s and Turner’s factors in terms of scientific software development.

Size

Scientific software development teams are typically small (Boisvert & Tang, 2001). Boehm and Turner (2005) characterize this as a good match to agile methods. They also

comment that the reliance on tacit knowledge also matches better to agile methods. Certainly, the difficulty in sharing scientific knowledge emphasizes the role of sharing tacit knowledge amongst a small team. Small teams of scientists often share understanding of existing theoretical background material for the software project, reducing the need for additional project-specific documentation.

Criticality

Scientific software is used to generate data for human decision-making. The human is in the system loop and is responsible for decisions, unlike embedded software where there is no intermediary between computational results and action taken. The scientist is an experimentalist, making changes to the environment (in this case software) and contemplating the results. This fits with Boehm and Turner's (2005) profile of an agile project.

Personnel

Boehm and Turner (2005) describe the agile development personnel as "requir[ing] continuous presence of a critical mass of ... experts" (p. 55). The development of scientific software requires the involvement of the scientist as the "critical mass". Again, this fits well with agile methods.

Culture

Cultural characteristics of agile methods are described by Boehm and Turner (2005) as "thriv[ing] in a culture where people feel comfortable and empowered by having many degrees of freedom available for them to define and work problems. This is the classic craftsman environment, where each person is expected and trusted to do whatever work is necessary ..." (p. 49). This aptly describes the scientist who is highly trained in his/her field and is comfortable with modeling and coding. The customer who poses the scientific question then leaves the subsequent project decisions to the scientist certainly expects the scientist to fulfill the role of empowerment.

Dynamism

Boehm and Turner (2005) do not define dynamism but measure it by "%requirements change per month" (p. 56). They suggest that "simple design and continuous refactoring [agile practices] are excellent for highly dynamic environments" (p. 55). This is the factor where development of scientific software fits neither agile nor plan-driven methods fully.

For scientific software, discussing requirements provides little help in the area where considerable time must be spent: exploring the solution space. The measure "%requirements change" is too difficult or impossible to assess. Yet, certain decisions must be made early in the project and get "locked in".

Development is certainly dynamic as scientists make changes to the software to explore and refine the software to better represent their science. In this way, scientists are risk-takers, but the risk-taking is limited and controlled. Changes are made as necessary to explore and improve the answers to the scientific question. To reduce collateral damage to the science, software changes for any other reason are discouraged (Boisvert & Tang, 2001). Refactoring, a defining XP activity, would be considered counter-productive to scientific development since it introduces unnecessary changes to the environment the scientist is studying. Refactoring software for a scientist would be equivalent to rearranging a bench-scientist's experimental set-up because it's ugly looking. This is not to say that simple, uncluttered code is not a good idea. However, working code for the scientist may mean leaving code in a state that would be considered less than ideal for the purpose of maintainability, say.

We can observe that plan-driven processes seek to augment exploration of the problem domain by extensive requirements and design planning and documentation. The assumption is that the complexities of the implementation step are thus reduced, possibly to the point of being pro forma. Before developing scientific software, the scientist usually relies on theory and mathematics that have been through sig-

nificant development and are well documented. This is part of the scientist's requirements and appears like plan-driven development. However, the scientist now changes mode and must spend significant time exploring the solution space of the software because of its inherent complexities and interactions. It is in this mode that the scientist appears to enter an agile development phase.

One of the goals of agile methods is to encourage exploration of the problem space, but with direct customer involvement as opposed to extensive documentation. When the scientist enters the agile-like part of the development phase, it is to explore the solution space. The customer may not be a useful participant at this point.

Another goal of agile methods is to address schedule pressures by dividing the problem into small pieces and delivering working code for these pieces. The solution space exploration phase of the scientist often requires a complete solution, obviating the division of the product. This exploration unfortunately can take considerable time and obtaining a correct answer will not yield to schedule pressures.

Even though, by Boehm's and Turner's model, scientific software development appears to be compatible with agile development, there are significant differences in underlying needs and assumptions. These are explored further by considering specific practices associated with one of the agile methods, XP.

4.5. Using Beck's Twelve Practices of XP as an Analysis Model

The popularity of agile approaches leads us to consider one of the approaches, Kent Beck's XP, in more detail. We consider the twelve practices that Beck (2000) uses to define XP to compare to scientific software development.

To start, we remove any practices whose statements are not refutable. For example, maintaining that a design should be as simple as possible is not refutable. Its opposite is that the design should be as complicated as possible. As a design philosophy, the statement about

simplicity is not useful for comparisons. Similarly, Beck's statement about limiting work to forty hours per week is desirable management of knowledge workers in any situation. This statement is also removed from our comparison.

The statement about coding standards is also problematic for comparisons. The reason cited for having coding standards is increased communication through the code base. Increased communication is always desirable. Using the code base as a means of communication also has a number of desirable side effects. The choice of software development method has no impact on whether this practice is used or not, and the practice itself is essentially not refutable. This is removed from our comparison.

From the remaining nine XP practices, the three overriding considerations appear to be speed of delivery of the product, communication during development, and flexibility for making possibly sweeping changes. We consider assumptions behind each of the nine practices of XP against the overriding considerations and environment characteristics of scientific software development.

We use three sets of experiences to comment on the nine remaining XP practices. Table 2 summarizes our comparisons.

The first set of experiences is our own through personal industrial work and interviews with other scientists (Kelly et al., 2004, 2006, 2008, 2010; Sanders & Kelly, 2008).

The second is an experience report (Ackroyd et al., 2008) following the adoption of XP practices at a research facility that developed data acquisition software for their own use. This software does not entirely fit our definition of scientific software since it includes a graphical user interface, and interfaces to hardware devices. However, there are some similarities that can be extracted. We designate this case study as *Tools*.

The third is a description of the introduction of agile methods into an academic environment where computational software is developed to model the electrical activity of the heart. The software uses large meshes and small time steps to solve systems of ordinary

Table 2. Practices that define XP versus their use with scientific software development

<i>XP Practice defined by Beck (2000, p. 54)</i>	<i>Scientific Software Development</i>
The Planning Game – quickly determine the scope of the next release ...	Major decisions are made up-front, more characteristic of a plan-driven approach.
Small releases – Put a simple system into production quickly, then release short versions on a short cycle.	Small releases generally are an antithesis to the indivisibility of the scientific problem being solved.
Metaphor – Guide all development with a simple shared story.	The simple shared story is the problem statement. The difficulty is the complexity of the solution domain.
Testing – Programmers continually write unit tests ... Customers write tests demonstrating that features are finished.	Good testing practices are always desirable. Customers usually cannot test the software without being familiar with the scientific domain.
Refactoring – Programmers restructure the system without changing its behaviour ...	Can be problematic with the complex interactions of techniques, compiler, hardware, and data. Best practices that govern the restructuring may not translate over to scientific software.
Pair programming – All production code is written by two programmers at one machine.	It may be that only one scientist has the knowledge necessary to address the scientific question. Alternate ideas to pair programming such as testing buddies and peer review may work in special cases.
Collective ownership – anyone can change code anywhere in the system at any time.	The combination of the above comments listed under “refactoring” and “pair programming” make this definition of collective ownership untenable.
Continuous integration – integrate and build the system many times a day, every time a task is completed.	Development of scientific code often involves long periods of careful consideration – thought time – and changes may not be rapid. The idea of tracking code changes in some manner is non-refutable.
On-site customer – include a real, live user on the team, available full-time to answer questions.	Customers can cover a spectrum of involvement: the scientist is his/her own customer; the customer provides the scientific question and empowers the scientist to pursue it; the customer is himself a scientist and may participate in the development and/or testing of the software.

differential equations. The software is run on parallel computers with the complications of efficiency and complex hardware architectures. The authors state that the overriding concern still is correctness of their code. They are dealing with large turnover in personnel and changes in technology and underlying science. They introduced agile techniques into a four-week course, and then expanded their experience with agile methods by developing a set of maintainable libraries over a three year period. Their experience is included in the discussions below, with the designation *HeartSim*.

The Planning Game – quickly determine the scope of the next release ... An assumption is made here that the problem being solved is highly divisible. We have found this is often not the case with scientific software. Generally, considerable planning has to be done to determine theoretical approaches, solution techniques, data requirements, and so on, long before any code is written. This is more characteristic of plan-driven approaches. The case study for *Tools* described the difficulty in getting users to formalize requirements and the need for developers to group requirements into regu-

lar releases. Even then, they had problems sticking to release dates. *HeartSim* did not discuss planning specifically.

Small releases – Put a simple system into production quickly, then release short versions on a short cycle. This practice makes the same assumption as the previous practice and suffers from the same misalignment with scientific software development. The description from *Tools* stated that evolutionary development fits the scientific research environment without further comment or details. We agree that evolutionary development fits the need to explore the solution space and iteratively refine the interactions of different parts of that space. However, this refinement is driven by correctness and trust, not by delivery time. The difficulties described by the *Tools* case study may have come from the failure to see this mismatch.

HeartSim also agreed that the iterative approach to development seemed to fit that of scientific software, but they categorize releases into external and internal. External releases were not tied to time boxing for the customer as assumed by XP, but to publishing an academic paper. These are large indivisible efforts. Internal releases to close collaborators allow more of an XP-like approach, but *HeartSim* did not discuss time frames for releases, but mentioned the need to have working code.

Metaphor – Guide all development with a simple shared story. There is some alignment with scientific software development, where the simple shared story is the scientific question to be answered. However, the story becomes very complex as the details are worked out. Who these details are shared with depends on the players in the development. But it is these details that are very important and the simple shared story provides little help with the details. If we consider the simple shared story as a management device for focusing the project, then this again becomes a non-

refutable statement. Neither the *Tools* case study nor *HeartSim* mention metaphors.

Testing – Programmers continually write unit tests ... Customers write tests demonstrating that features are finished. Good testing practices are another non-refutable statement. But the interesting assumption here is the role of the customer. In our experience, unless the customer is intimately familiar with the scientific domain of the software, the customer cannot write and evaluate acceptance tests. The *Tools* case study simply said that they needed to improve their testing practices. *HeartSim* discussed the different types of modules typically found in scientific software, where some are easy to test and others extremely difficult. They obviously agreed that good testing techniques and practices are beneficial, and it was also obvious that a great deal of technical knowledge was needed to do their testing, not a role assumed by a non-technical customer.

Refactoring – Programmers restructure the system without changing its behaviour ... Refactoring is defined as restructuring the system without changing its behaviour in order to increase desirable properties of the code. Again, this appears like a non-refutable statement: a globally good thing to do. The assumption behind this is that anyone on the team can change the code anywhere at any time. Discussions with some scientists have raised some very strong reactions against this practice. The interactions of code with compilers, solution techniques, hardware, and data can create a complex situation where the scientist becomes very conservative in their desire to change the code. Anyone who makes a code change needs to have a full understanding of these interactions. In addition, “best practices” have been usually defined for software other than scientific software. In one case study, the practice of breaking modules up to reduce their size actually reduced the potential

comprehensibility of the software for the scientific programmer (Kelly, 2006). The *Tools* case study only commented that the automatic refactoring provided by an integrated development environment was helpful. *HeartSim* did not specifically discuss refactoring.

Pair programming – *All production code is written by two programmers at one machine.*

The assumption is that programmers have overlapping knowledge, though possibly differences in experience. The specialized knowledge of scientists writing scientific software is often spread very thinly and would make the idea of two highly specialized scientists sitting together to work on one code a rarity. The only situation where we have seen something similar to this idea at work is with a scientific specialist who did not write code and a scientist non-specialist who did. The non-specialist had become extremely familiar with the specialized field, worked closely with the specialist, and wrote the code in such a way that it was “readable” by the specialist. Pair programming in this case transformed into pair peer reviews, sitting together at the terminal and reading the code. The *Tools* case study found it was difficult to enforce pair programming, for various reasons. *HeartSim* enforced pair programming in group development sessions held one day a week. Outside those sessions, all development was solo. Almost all code developed by students and researchers for their own projects was done solo. However, some projects were peer-reviewed, adding back some of the benefits of pair programming.

Collective ownership – *anyone can change code anywhere in the system at any time.*

This practice makes the same assumptions as refactoring and suffers from the same limitations when applied to scientific software development. The *Tools* case study made attempts at enforcing collective ownership, but particular project pressures

resulted in individuals taking ownership of specific parts of the code. As discussed by *HeartSim*, collective ownership in an academic environment is problematic in many cases. For the core libraries, they discussed mechanisms such as peer review and change-set tracking to maintain some on-going expertise in key areas of code in the face of large personnel turnover. The concern is not ubiquitous change by anyone at any time, but the maintenance of corporate memory to do changes over long time periods. This speaks to the much longer time scale that is typically of concern with scientific software.

Continuous integration – *integrate and build the system many times a day, every time a task is completed.*

The underlying philosophy of being organized, frequently checking code changes, testing frequently, and maintaining reproducibility is, again, non-refutable. The problem in the statement of this practice is “many times a day”. The focus is on speed of delivery rather than careful consideration of problems, alternatives, and next steps. This practice also suggests a much less complex solution space than the scientists are dealing with. The *Tools* case study agreed that continuous integration was helpful though sometimes difficult to enforce. Their comment on time came from an attempt to implement daily meetings. Their change to weekly meetings worked better and also fits with our observations that a longer time box is required for scientific software. Continuous integration is not discussed by *HeartSim*.

On-site customer – *include a real, live user on the team, available full-time to answer questions.*

The customer is often the scientist who is writing the software. In other cases, the customer may have devolved all responsibility for making key decisions to the scientist, so the customer’s involvement is minimal. In other cases that we have seen, the customer is another scientific

group co-located with the scientists who are writing the software. Their interactions are typically informal, frequent, and very detailed, often sharing both code writing and testing. The underlying philosophy of this practice is on-going, meaningful communication with the customer, which is again non-refutable. The means of promoting this practice is highly dependent on the situation. The *Tools* case study mentions team collaborations but doesn't specifically say if they had on-site customers. *HeartSim* discusses the wide variation in roles for the customers and agrees with our experience that how this is handled is highly dependent on the situation.

5. SUMMARY

The discussions in this paper have exposed a number of mismatches between the characteristics of scientific software and the assumptions behind both plan-driven and agile software development methodologies.

Both categories of methodologies make assumptions about time required to explore problem space as opposed to solution space. These assumptions are at odds with scientific software development. For scientific software, the theory exploration as a part of the problem domain is usually previously established and documented. The overwhelming need is to dynamically explore the solution space because of complex interactions of elements of the solution. This often requires large portions of the code to be assembled before this exploration can take place. This is also a process that is not amenable to the participation of a non-specialist customer.

The highest priority concern with scientific software is trustworthiness. The focus of agile methods is mainly speed of delivery of the product. It can be argued that plan-driven methods are more aligned with trust in the product rather than speed of delivery.

Time factors coupled with knowledge acquisition play a role in the mismatch. There

are three cycles of knowledge acquisition in scientific software development that evolve at different rates. There is a short cycle of learning about the software and execution environment that lends itself to agile timelines. There is a long cycle of learning about the final product and its relation to the exterior environment that could line up with plan-driven timelines. However, there is an intermediate cycle of learning about the interactions of software, hardware, and theory that is not aligned with either agile timelines or plan-driven timelines. This is an important cycle of learning and plays havoc with the application of either category of methodology.

Both categories of methodologies assume the customer drives out the detailed requirements of the project. With scientific software, the customer may simply pose the problem, that is the "metaphor", and leave details to the scientist.

Scientific software is positioned between plan-driven and agile methodologies in respect to changes to the code. Before implementation begins the scientist invariably starts with a developed theory and plan. There will be parts of these plans that will be immutable, such as using particular equations of state and a matrix solution technique. As the intermediate cycle of learning begins, changes will be made to the code. At this point, development appears to be iterative as in agile approaches. However, it is not schedule-based, nor is it divisible. As well, changes are contained and focused on the exploration. This "phase" of scientific software development fits neither plan-driven nor agile approaches.

The goal of XP's refactoring is to allow anyone anytime to improve the quality of the code base. Unfortunately, quality is in the eyes of the beholder, and many qualities considered "best practices" for other types of software have not been validated for scientific software. Qualities important to the scientist are different from those typically considered important to the software engineer.

The time constant for scientific software is much longer than that assumed by agile methodologies. Understanding a change may

take weeks rather than hours. As well, the lifetime of scientific software is often decades. This requires that learning be preserved over a number of years and over a large number of people disassociated by time. The focus of agile is sharing knowledge within the initial development of the product with temporally coincident participants. Unfortunately, plan-driven methods also fail to address this problem, with documents focused on describing the problem space of the product. Scientists already have a long and well-established history of documentation of their science. This is essentially their systems requirements. Their need is for documentation that supports long-term evolution of their products. All this points to a need for research into viable methods of preserving and transferring knowledge over time for scientific software products and for extending documentation scientists already produce as part of this knowledge transfer.

6. CONCLUSION

We define scientific software as the core part of the scientific product exclusive of user and hardware interfaces and of support libraries. The characteristics of the development of scientific software as we defined it are such that they do not fully mesh with the assumptions of either plan-driven or agile software development methodologies. These fundamental mismatches demonstrate the unsuitability of whole-heartedly using either type of process. There is a danger of imposing activities that detract from fundamental exploration that must be done and substituting values that do not fit with the goals of scientific software. Problems are already arising where quality standards wholly based on plan-driven development methodologies have been enforced on the development of scientific software in particular application areas.

This mismatch would also strongly argue for the careful partitioning of projects so that user interfaces, for example, are developed separately from the scientific part of the product. Development of other parts of the product more likely matches the assumptions inherent in agile

and plan-driven methodologies. Certainly, the time lines, quality goals, and knowledge domains are very different when we compare the scientific core of the product to the creation of the user interface. This supports the argument for careful project partitioning.

There is considerable interest in research into the application of agile methodologies to scientific software development. Understanding the dynamics of developing scientific software will help to profitably move this research forward. Dynamics include the interaction of time, knowledge, software quality, and goals. Once that understanding is sufficient, we can assess the value of particular activities that comprise a suitable software development methodology.

Knowledge transfer over an extended period of time and across possibly temporally disassociated contributors to the software product is a major problem with the long-term evolution and use of scientific software. This problem is in addition to the knowledge transfer amongst co-temporal contributors as the product is developed. This is an important research area that would contribute to the quality of the scientific code bases.

There are irrefutable good software goals that all software developers should consider. However, successfully weaving activities to achieve these goals into a process that supports the needs of scientists writing software is not easy. We need to consider factors not traditionally included in software engineering processes, factors that, so far, seem to be unique to the development of scientific software.

REFERENCES

- Ackroyd, K. S., Kinder, S. H., Mant, G. R., Miller, M. C., Ramsdale, C. A., & Stephenson, P. C. (2008). Scientific software development at a research facility. *IEEE Software*, 25(4), 44–51. doi:10.1109/MS.2008.93
- Basili, V., Cruzes, D., Carver, J., Hochstein, L., Hollingsworth, J., Zelkowitz, M., & Shull, F. (2008). Understanding the high-performance-computing community: A software engineer's perspective. *IEEE Software*, 25(4), 29–36. doi:10.1109/MS.2008.103

- Beck, K. (2000). *Extreme programming explained*. Reading, MA: Addison-Wesley.
- Beck, K. (2003). *Test-driven development by example*. Reading, MA: Addison-Wesley.
- Boehm, B., & Turner, R. (2005). *Balancing agility and discipline*. Reading, MA: Addison-Wesley.
- Boisvert, R. F., & Tang, P. T. P. (Eds.). (2001). *The architecture of scientific software*. Boston, MA: Kluwer Academic.
- Hatton, L. (2007). The chimera of software quality. *IEEE Computer*, 40(8), 102–104.
- Hatton, L., & Roberts, A. (1994). How accurate is scientific software? *IEEE Transactions on Software Engineering*, 20(10), 786–797. doi:10.1109/32.328993
- Henderson-Sellers, B. (2003). Method engineering for OO system development. *Communications of the ACM*, 46(10), 73–78. doi:10.1145/944217.944242
- Highsmith, J., & Cockburn, A. (2001). Agile software development: The business of innovation. *IEEE Computer*, 34(9), 120–122.
- Hook, D., & Kelly, D. (2009). Testing for trustworthiness in scientific software. In *Proceedings of the 2nd International Workshop on Software Engineering for Computational Science and Engineering*, Vancouver, BC, Canada.
- Jackson, M. (1994). Problems, methods, and specialisation. *IEEE Software Engineering Journal*, 9(6), 249–255.
- Jackson, M. (2001). *Problem frames*. Reading, MA: Addison-Wesley.
- Jeffries, R., Anderson, A., & Hendrickson, C. (2001). *Extreme programming installed*. Reading, MA: Addison-Wesley.
- Kelly, D. (2006). A study of design characteristics in evolving software using stability as a criterion. *IEEE Transactions on Software Engineering*, 32(5), 315–329. doi:10.1109/TSE.2006.42
- Kelly, D. (2008). Innovative standards for innovative software. *IEEE Computer*, 41(7), 88–89.
- Kelly, D. (2009). Determining factors that affect long-term evolution in scientific application software. *Journal of Systems and Software*, 82, 851–861. doi:10.1016/j.jss.2008.11.846
- Kelly, D., & Shepard, T. (2004). Task-directed inspection. *Journal of Systems and Software*, 73(2), 361–368. doi:10.1016/j.jss.2003.10.027
- Kelly, D., Thorsteinson, S., & Hook, D. (2010). Scientific software testing, analysis with four dimensions. *IEEE Software*, 28(3), 84–90. doi:10.1109/MS.2010.88
- Matthews, D., Wilson, G., & Easterbrook, S. (2008). Configuration management for large-scale scientific computing at the UK Metoffice. *IEEE Computational Science & Engineering*, 10(6), 56–64.
- Miller, G. (2006). Scientific publishing: A scientist's nightmare: Software problem leads to five retractions. *Science*, 314(12), 1856–1857. doi:10.1126/science.314.5807.1856
- Pearson, C. (2009). *The demise of the iron triangle*. Retrieved February 19, 2010, from <http://mosaicprojects.wordpress.com/2009/11/19/the-demise-of-the-iron-triangle/>
- Pfleeger, S. L. (1998). *Software engineering theory and practice*. Upper Saddle River, NJ: Prentice Hall.
- Pitt-Francis, J., Bernabeu, M. O., Cooper, J., Garny, A., Momtahan, L., & Osborne, J. (2008). Chaste: Using agile programming techniques to develop computational biology software. *Philosophical Transactions of the Royal Society*, 366, 3111–3136. doi:10.1098/rsta.2008.0096
- Post, D., & Kendall, R. (2004). Software project management and quality engineering practices for complex, coupled, multiphysics, massively parallel computational simulations: Lessons learned from ASCI. *International Journal of High Performance Computing Applications*, 18(4), 399–416. doi:10.1177/1094342004048534
- Rawsthorne, D., & Shimp, D. (2009). *Scrum in a nutshell*. Retrieved February 19, 2010, from <http://www.scrumalliance.org/articles/151-scrum-in-a-nutshell>
- Rolland, C. (1997). A primer for method engineering. In *Proceedings of the IFIP TC8, WG8.1/8.2 Working Conference on Method Engineering: Principles of Method Construction and Tool Support*, Toulouse, France (pp. 1-7).
- Royce, W. (1970, August). Managing the development of large software systems. In *Proceedings of the 9th International Conference on Software Engineering* (pp. 328-338).

- Sanders, R., & Kelly, D. (2008). Scientific software: Where's the risk and how do scientists deal with it? *IEEE Software*, 25(4), 21–28. doi:10.1109/MS.2008.84
- Segal, J. (2004). *Professional end user developers and software development knowledge* (Tech. Rep. No. 2004/25). Milton Keynes, UK: Open University.
- Segal, J. (2005). When software engineers met research scientists: A case study. *Empirical Software Engineering*, 10(4), 517–536. doi:10.1007/s10664-005-3865-y
- Song, X., & Osterweil, L. J. (1992). Toward objective systematic design-method comparisons. *IEEE Software*, 9(3), 43–53. doi:10.1109/52.136166
- Van Vliet, H. (2000). *Software engineering principles and practices*. Chichester, UK: John Wiley & Sons.
- Weyuker, E. (1982). On testing non-testable programs. *The Computer Journal*, 25(4), 465–470.
- Wiegers, K. E. (1999). *Software requirements*. Redmond, WA: Microsoft Press.
- Wood, W. A., & Kleb, W. L. (2003). Exploring XP for scientific research. *IEEE Software*, 20(3), 30–36. doi:10.1109/MS.2003.1196317

Diane Kelly is an associate professor at the Royal Military College of Canada (RMC). She has over twenty years of industrial experience in scientific software development and has spent the past ten years combining academic research with what she's learned from her industrial experience. Dr. Kelly has a BSc in pure mathematics and BEd in mathematics and computer science, both from the University of Toronto. Her MEng and PhD are in software engineering, both from RMC. She is a member of ACM and a senior member of IEEE.